School of Computer Science, UPES, Dehradun

A Project Report on

Vision-Guided Sorting Arm on a Conveyor

Submitted for the course

CSAI4014 Algorithms for Intelligent Systems and Robotics

B.Tech CSE (AI & ML), Semester VII

Project No.: P14
Group No.: G10
Batch: B4

Submitted by

S. No.	Name	SAP ID
1	Aditya Khatri	500119453
2	Simran Jeet Gill	500122195
3	Divya Israni	500121919
4	Amritanshi Singh	500119477
5	Anchit Saxena	500122868

Submitted to

Course Faculty, CSAI4014

School of Computer Science, UPES, Dehradun

Academic Year 2026-2027 | October 2026

Declaration

We, the undersigned members of Group G10 (Batch B4), declare that the project report entitled "Vision-Guided Sorting Arm on a Conveyor" is an original piece of work carried out by us for the course CSAI4014 Algorithms for Intelligent Systems and Robotics. The simulation code, experiments, figures and text were prepared by the group; every external idea, equation, dataset or figure has been acknowledged and referenced. The report has not been submitted elsewhere, and no part of it has been copied from another report or from the internet without citation. We understand that plagiarism leads to cancellation of the project marks as per university rules.

Member	**SAP ID**	**Signature**
Aditya Khatri	500119453	
Simran Jeet Gill	500122195	
Divya Israni	500121919	
Amritanshi Singh	500119477	
Anchit Saxena	500122868	

Date: _____ Place: Dehradun

Abstract

Quality inspection and sorting on moving conveyors is common in manufacturing, but a robot that must pick moving parts often needs the belt stopped, which wastes throughput. This project builds a vision-guided sorting arm in CoppeliaSim, the assigned platform, and validates the same software on a second physics engine, MuJoCo. A vision sensor detects parts by colour and shape: red, green and blue cubes go to colour bins and cylinders, which stand for wrong-shape parts, go to a reject bin. A SCARA arm with a suction gripper intercepts each part while the belt moves. The Python and OpenCV software contains a colour and shape detector, a Kalman tracker that fuses camera and belt-encoder data, a D-H based inverse-kinematics intercept planner with minimum-jerk motion, and joint PID control with feedforward. The novelty is a predictive visual-servo descent that keeps correcting the grasp point while the arm moves. In 560 MuJoCo runs it cut the mean grasp error from 4.2 to 3.2 mm and kept 88 % success at 15 % belt-speed ripple against 49 % for plain prediction; 105 CoppeliaSim runs confirm the ranking in 4 of 4 speeds.

Keywords: vision-guided sorting, quality inspection, SCARA arm, D-H transformations, visual servoing, CoppeliaSim, MuJoCo, OpenCV.

Index

Declaration i

Abstract ii

Index iii

List of Figures iv

List of Tables iv

Writing Allocation v

1. Introduction 1

2. Problem Statement and Objectives 2

3. Literature and Industry Review 4

4. Novelty 5

5. Syllabus Mapping and Theory 6

6. System Design 10

7. Methodology and Algorithms 12

8. Simulation Setup 14

9. Results and Analysis 17

10. Contribution 24

11. Co-learning Reflection 24

12. Conclusion and Future Scope 25

References 26

Appendix: Links and Logs 27

List of Figures

Figure 1: Top view of the sorting cell (belt, camera view, pick window, arm, bins). 3
Figure 2: Block diagram of the complete system. 10
Figure 3: The simulated cell: (a) CoppeliaSim (assigned platform) and (b) MuJoCo; SCARA arm, conveyor, three colour bins and the dark reject bin. 10
Figure 4: Flowchart of the proposed method (M3), from image capture to bin release. 12
Figure 5: (a) Camera frame rendered by MuJoCo and (b) detected blobs with colour, shape and centroid. 13
Figure 6: Joint 1 controller: (a) step response; (b) tracking error (solid: MuJoCo, dashed: CoppeliaSim). 17
Figure 7: Vision accuracy versus image noise for three lighting levels. 18
Figure 8: (a) Sorting success, (b) throughput and (c) grasp error versus belt speed. 19
Figure 9: Success at 0.25 m/s versus latency, ripple and image noise. 20
Figure 10: Same trial for M2 and M3: approach and error at contact. 21
Figure 11: Snapshots of one trial of the proposed method. 21
Figure 12: Ablation: (a) sorting success and (b) grasp error of the variants. 22

List of Tables

Table 1: Writing allocation of the report. v
Table 2: Measurable objectives and the results that verify them. 3
Table 3: Comparison of existing approaches with the approach of this project. 5
Table 4: Novelty points and the evidence that supports them. 6
Table 5: AISR syllabus topics of Project P14 and where they are used. 7
Table 6: Denavit-Hartenberg parameters of the SCARA arm. 7
Table 7: Principal parameters of the simulated cell. 11
Table 8: The three strategies and the ablated variants. 14
Table 9: Elements of the CoppeliaSim set-up named in the Project List and their implementation. 15
Table 10: Files of the repository and commands to reproduce all results. 16
Table 11: Validation checks of the simulation. 17
Table 12: Results versus belt speed (mean of 10 runs with 15 objects each). 19
Table 13: Ablation at 0.30 m/s, 6 % ripple, 160 ms latency (10 runs, 15 objects each). 21
Table 14: Confirmation study of the ablation with 30 new seeds (450 parts per variant). 22

Table 15: M3 versus M2 in the confirmation study (30 new seeds). 22
Table 16: Speed study in CoppeliaSim (5 seeds, 12 parts) and in MuJoCo (10 seeds, 15 parts). 23
Table 17: Technical contribution of each member and report sections written. 24
Table 18: Meeting log of the group. 28
Table 19: Peer-assessment sheets (one per member). 28

Writing Allocation

The writing of this report was divided equally among the five members (Table 1). Each member wrote the sections listed below; every member also wrote the own reflection in Section 11, filled the own row (with signature) in Section 10, and signed the declaration.

Table 1: Writing allocation of the report.

Member	Name and SAP ID	Sections written	Pages
M1	Aditya Khatri (500119453)	Abstract, 1, 2, 12, Appendix	6.0
M2	Simran Jeet Gill (500122195)	3, 4, 8	6.0
M3	Divya Israni (500121919)	5, 6	5.5
M4	Amritanshi Singh (500119477)	7, 9.1-9.2, References	5.0
M5	Anchit Saxena (500122868)	9.3-9.8	5.5
All	Every member	10 (own row and signature), 11 (own reflection), Declaration	-

1. Introduction

1.1 Background

Sorting objects that travel on a conveyor belt is one of the most common applications of industrial robots. Parcel hubs sort packages by destination, electronics factories separate components by type, food plants grade produce by colour and recycling plants pick plastics from a moving stream. In all these cases a camera tells the robot what and where the object is, and the robot must pick it before it leaves the working area. Commercial delta and SCARA robots with line-tracking software reach picking rates of the order of 100 objects per minute [1], [2], [3], which is far above what a human operator can do.

Two difficulties make the problem harder than picking a stationary object. First, the object moves between the moment it is photographed and the moment the gripper touches it. The camera image is processed with a delay of tens of milliseconds, the arm needs a fraction of a second to move, and the belt speed itself is never perfectly constant. Second, the sensing is noisy: lighting changes, image noise and calibration errors shift the measured position by millimetres, which is already comparable to the size of a suction cup. A robot that aims at the position where the object was seen therefore misses it.

1.2 Why it matters in industry

The simplest industrial remedy is to stop the conveyor while the robot picks, or to place a mechanical fence that holds the object. This works, but it limits throughput, wears the belt drive and complicates the layout. Continuous line tracking removes these restrictions and is therefore a key feature of modern pick-and-place cells. Making line tracking accurate and robust against latency, belt-speed fluctuation and sensor noise decides how fast a cell can run and how many mis-picks occur. Even a few millimetres of improvement in grasp accuracy allows a smaller suction cup, a higher belt speed or a lower reject rate.

1.3 Application domain: quality inspection and sorting in manufacturing

The application domain of this project is quality inspection and sorting in manufacturing. On an assembly or packaging line, a camera checks every part that passes: the colour identifies the variant (for example the colour code of a component or of a cap) and the outline identifies whether the part has the correct shape. A part with the wrong outline is a reject and must be removed from the flow, while correct parts must be delivered to the bin of their variant. In our cell the correct parts are cubes in three colours and the rejects are cylinders (25 % of all parts). A robot that does this on a moving belt without stopping it is

attractive because every stop of the belt reduces the output of the whole line. The inspection quality is measured by two error rates: false accepts (a reject delivered to a colour bin) and false rejects (a correct part sent to the reject bin).

1.4 Scope of this project

This project (Project P14 of the AISR course) builds a complete vision-guided sorting arm on a conveyor in simulation. The system recognises parts by colour and shape, tracks them, plans an intercept for a SCARA arm, executes the motion with a joint-level controller and places each cube into the bin of its colour and each wrong-shape part into the reject bin. Because no physical robot is available, the cell is built in CoppeliaSim [4], the platform assigned to this project (vision sensor, arm and conveyor, controlled from Python with OpenCV). The same software also runs on a second physics engine, MuJoCo [5], which is used for the large statistical studies. The report presents the design, the algorithms, a quantitative comparison of three strategies, stress tests on latency, speed ripple and image noise, and a cross-platform validation. The remaining sections follow the format of the course: problem statement, literature review, novelty, theory, design, method, simulation setup, results, contributions, reflections, conclusion and references.

2. Problem Statement and Objectives

2.1 Problem definition

A conveyor moves at a nominal speed v between 0.05 and 0.35 m/s. Parts of height 30 mm are placed at random lateral positions and orientations with a random spacing of 0.35 to 0.55 m: 75 % are cubes (side 30 mm) in red, green or blue and 25 % are cylinders (diameter 36 mm) in any of the three colours. A fixed overhead camera observes the belt. A SCARA arm with a suction gripper stands beside the belt (Figure 1). The task is to detect every part by colour and shape, pick it while the belt keeps moving and to release cubes into the bin of their colour and cylinders into the reject bin, with the highest possible success rate and throughput. Figure 1 shows the layout, the camera field of view, the pick window and the four bins.

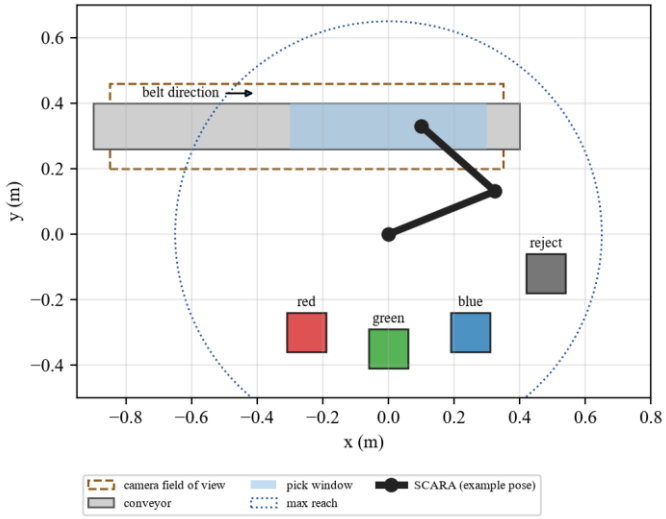


Figure 1: Top view of the sorting cell (belt, camera view, pick window, arm, bins).

2.2 Measurable objectives

The project has five measurable objectives. Table 2 lists them with the target and the result that verifies each one in Section 9.

Table 2: Measurable objectives and the results that verify them.

No.	Objective	Target	Achieved (see Section 9)
O1	Detect parts and classify colour and shape in the camera image	>= 99 % correct at image noise up to 25 gray levels (3 x the default)	colour 100 %, shape 100 % (noise 25, nominal light)
O2	Grasp accuracy with the moving belt	mean tip-to-object error <= 5 mm up to 0.35 m/s	2.7 to 3.4 mm (M3)
O3	Sorting success and robustness	>= 95 % at v <= 0.20 m/s, and within 10 points of that when belt ripple rises from 3 % to 15 %	100.0 % at 0.20 m/s; 88.0 % (15 % ripple) vs 93.3 % (3 %) (M3)
O4	Throughput gain over stop-and-go	>= 2 x at 0.25 m/s	2.2 x (28.5 vs 13.2 obj/min)
O5	Inspection quality: wrong accepts and rejects	false accepts <= 0.1 % and false rejects <= 0.1 % at image noise up to 40	false accepts 1 of 3213 cylinders (0.03 %), false rejects 0 of 9837 cubes

2.3 Assumptions and limits

- The belt is planar and moves along the x-axis; the parts ride the belt without slipping (the conveyor is kinematic: the parts are forced to the belt speed). The belt speed carries a sinusoidal ripple (default 3 %, tested up to 15 %), and the encoder reading has 3 mm/s noise.
- The camera is an overhead orthographic vision sensor (no lens distortion) rendered from the 3-D scene at 1.5 mm/pixel, 20 frames per second and a total latency of 80 ms. The arm can hide parts from the camera; the carried part is masked in the image.
- The arm is a rigid-body model with link masses, inertias and torque limits (45 N m and 18 N m); the vertical axis is ideal in CoppeliaSim and position-controlled in MuJoCo.

The suction gripper is modelled as an attach constraint (CoppeliaSim: the part becomes a child of the tool tip; MuJoCo: a weld) that is switched on when the tip is within 10 mm of the part centre and off at the bin.

- Parts are free rigid bodies with contact and friction; a part counts as sorted only if it physically comes to rest inside the correct bin (cubes: colour bin, cylinders: reject bin). Cylinders stand for defective, wrong-shape parts and appear with a probability of 25 %.
- Only three colours and two shapes are used and objects never touch each other on the belt. Real-robot issues such as cable slack, vibration, vacuum dynamics and collision avoidance with other equipment are listed as future work.

3. Literature and Industry Review

This section reviews the research and the industrial practice that the project builds on, and identifies the gap that the project addresses.

3.1 Robot kinematics, trajectories and control

The kinematics of serial manipulators, including Denavit-Hartenberg parameters, inverse kinematics and the Jacobian, are standard textbook material [6], [7]. SCARA robots are a planar two-link arm with a vertical axis and are widely used for fast pick-and-place, because their closed-form inverse kinematics and high horizontal stiffness allow very short cycle times. Minimum-jerk trajectories, proposed to describe human arm movements by Flash and Hogan [8], give smooth quintic motion with zero start and end velocity and acceleration and are common in robot motion planning. Joint-level PID control with model feedforward is still the dominant industrial servo structure [9].

3.2 Vision and visual servoing

Colour segmentation in HSV space, connected-component analysis and calibration with a planar homography are classical and efficient tools available in OpenCV [10], [11]. Visual servoing uses image measurements directly inside the control loop. Chaumette and Hutchinson [12], [13] describe position-based and image-based schemes and show that feedback from vision reduces the effect of calibration and model errors. Corke [14] gives a practical treatment that combines vision and robot control, including moving targets.

3.3 Estimation and tracking

The Kalman filter [15] is the standard method to estimate position and velocity of a moving target from noisy measurements. Bar-Shalom et al. [16] treat constant-velocity

tracking, data association and delayed measurements in detail. In conveyor applications the belt encoder gives an almost noise-free speed that can be fused with the camera.

3.4 Industrial products and standards

Industrial delta robots, introduced by Clavel [1], and SCARA robots are combined with line-tracking software in commercial products such as FANUC iRVision [2] and ABB PickMaster [3]. These systems read the belt encoder, trigger the camera at a fixed belt position and compensate the travelled distance. Performance of such robots is specified with standardised criteria such as positioning accuracy and repeatability in ISO 9283 [17]. Table 3 compares these approaches with the approach of this project.

Table 3: Comparison of existing approaches with the approach of this project.

Approach	Source	Strength	Gap addressed here
Stop-and-go conveyor	common practice	very accurate and simple	low throughput (about half in our tests)
Encoder-based line tracking	[2], [3]	high rates, industrial maturity	closed commercial systems; open comparison with ablations missing
Visual servoing theory	[12], [13], [14]	feedback reduces model error	rarely combined with intercept planning of a moving, delayed target in an open simulator
Kalman target tracking	[15], [16]	optimal for linear Gaussian models	benefit for grasping is rarely quantified against a line fit

3.5 Identified gap

Most open educational examples either stop the conveyor or aim at the last measured position, and commercial line-tracking software is closed. What is missing is a transparent, reproducible study that (i) combines a latency-compensated Kalman tracker, an intercept planner and a visual-servo descent in one pipeline, (ii) compares it with the stop-and-go and plain predictive baselines over belt speed, latency, speed ripple and image noise, and (iii) uses an ablation to show which component gives which benefit. This project provides that study.

4. Novelty

The work is not a new robot but a new combination and analysis that goes beyond the textbook pick-and-place example. Each point below is supported by measurements from Section 9; Table 4 summarises the evidence.

- **N1. Predictive visual-servo descent.** During the 0.2 s descent the tip is driven to the refined grasp point $\hat{p}(t_g)$ through the inverse Jacobian instead of holding the planned

point. At 0.30 m/s with 6 % ripple and 160 ms latency it lifted success from 64.0 % (without servo) to 78.0 % and cut the mean grasp error from 4.7 mm to 3.3 mm.

- **N2. Latency-compensated Kalman tracker with encoder fusion.** The belt encoder speed enters the filter as a pseudo-measurement together with a zero lateral velocity constraint. Against a least-squares line fit it raised success from 72.0 % to 78.0 % and lowered the grasp error from 4.50 mm to 3.32 mm in the same ablation (significant in the 30-seed confirmation study of Section 9.6). Its success stays at 93.3 % for every latency from 0 to 320 ms, while the line-fit method falls from 90.0 % to 76.7 %.
- **N3. Robustness to belt-speed ripple.** At 15 % ripple the proposed pipeline still sorted 88.0 % of the objects, against 48.7 % for the plain predictive method.
- **N4. A quantitative three-way study on an open, physics-based simulator.** A MuJoCo cell with a rendered camera, 80 ms latency, encoder noise, a torque-driven arm and rigid-body cubes was built, and 560 runs compare stop-and-go, plain predictive and the proposed method. Success is counted only if the cube physically rests in the right bin. The proposed method reaches 2.2 times the throughput of stop-and-go at 0.25 m/s.
- **N5. An honest negative result.** A shortest-cycle-time scheduling rule among feasible targets (SCT) was implemented and tested; it gave no measurable gain over first-in-first-out (78.0 % in both cases) because all objects move with the same speed. We report it as a negative finding instead of claiming it.

Table 4: Novelty points and the evidence that supports them.

Point	Evidence	Where
N1 Visual-servo descent	grasp error 4.7 -> 3.3 mm; success 64 -> 78 %	Table 13, Figure 12
N2 Kalman + encoder	success 70 -> 78 % (30 seeds, $p < 0.001$); flat versus latency	Table 14, Figure 9
N3 Ripple robustness	88 % vs 49 % success at 15 % ripple	Figure 9
N4 Three-way physics study	2.2 x throughput over stop-and-go at 0.25 m/s	Table 12, Figure 8
N5 SCT scheduling	no gain (78.0 % in both cases)	Table 13

5. Syllabus Mapping and Theory

Table 5 maps the AISR topics of Project P14, as named in the Project List, to the sections of this chapter.

Table 5: AISR syllabus topics of Project P14 and where they are used.

AISR topic (Project List)	Used for	Section
Robotic Perception	Rendered camera, homography calibration, HSV colour segmentation, contour-based shape classification, Kalman tracking	5.5, 5.6
D-H Transformations	D-H table and homogeneous matrices A1 A2 A3 of the SCARA arm, forward and inverse kinematics, Jacobian	5.1, 5.2
Closed-Loop Control	Joint PID with feedforward, minimum-jerk reference, visual-servo descent (vision in the loop)	5.3, 5.4
Application Domains	Quality inspection and sorting in manufacturing: colour and shape inspection, reject bin, throughput	1.3, 5.7

5.1 Kinematics of the SCARA arm

The arm has two revolute joints (q_1, q_2) with link lengths $L_1 = 0.35$ m and $L_2 = 0.30$ m and a prismatic vertical axis z. The Denavit-Hartenberg parameters [18], [7] are listed in Table 6. With the D-H convention each link is described by the homogeneous transformation $A_i = \mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i)$:

Table 6: Denavit-Hartenberg parameters of the SCARA arm.

Joint i	a_i (m)	α_i	d_i (m)	θ_i
1	0.35	0	0	q1
2	0.30	0	0	q2
3	0	0	-z	0 (prismatic)

$$A_i = [c\theta_i \;\; -s\theta_i c\alpha_i \;\; s\theta_i s\alpha_i \;\; a_i c\theta_i;\; s\theta_i \;\; c\theta_i c\alpha_i \;\; -c\theta_i s\alpha_i \;\; a_i s\theta_i;\; 0 \;\; s\alpha_i \;\; c\alpha_i \;\; d_i;\; 0\;0\;0\;1] \qquad (1)$$

For the SCARA table (all $\alpha_i = 0$) the chain $T_0^3 = A_1\,A_2\,A_3$ gives the pose of the tool frame:

$$T = [c_{12} \;\; -s_{12} \;\; 0 \;\; L_1c_1 + L_2c_{12};\; s_{12} \;\; c_{12} \;\; 0 \;\; L_1s_1 + L_2s_{12};\; 0\;0\;1\;-z;\; 0\;0\;0\;1], \quad c_{12} = \cos(q_1 + q_2) \qquad (2)$$

Its position column is the forward kinematics of the tip, and its rotation part shows that the tool turns by $q_1 + q_2$ about the vertical axis. A unit test confirms that the matrix chain equals the closed-form tip position for 100 random poses:

$$x = L_1 \cos q_1 + L_2 \cos(q_1 + q_2), \quad y = L_1 \sin q_1 + L_2 \sin(q_1 + q_2) \qquad (3)$$

For a desired tip point (x, y) the closed-form inverse kinematics gives two elbow solutions:

$$\cos q_2 = (x^2 + y^2 - L_1^2 - L_2^2) / (2 L_1 L_2), \quad q_2 = \pm \arccos(\cos q_2) \qquad (4)$$

$$q_1 = \mathrm{atan2}(y, x) - \mathrm{atan2}(L_2 \sin q_2,\; L_1 + L_2 \cos q_2) \qquad (5)$$

The solution within the joint limits ($|q_1| <= 2.97$ rad, $|q_2| <= 2.53$ rad) that is closest to the present joint angles is selected. A target outside the reachable annulus ($|c_2| > 1$) is rejected.

5.2 Jacobian and resolved-rate control

The velocity of the tip is related to the joint rates by the Jacobian J(q), with det J = $L_1 L_2 \sin q_2$. Away from the singularities ($q_2 = 0, \pi$), a desired tip velocity is converted to joint rates by:

$$dq/dt = J(q)^{-1} v_{tip} \qquad (6)$$

This relation drives the visual-servo descent in Section 7.5. In the workspace used here $|q_2|$ stays between 1.1 and 2.4 rad, so the Jacobian is well conditioned.

5.3 Minimum-jerk trajectory

Each point-to-point motion from q_0 to q_1 in time T uses the quintic blend [8]:

$$q(t) = q_s + (q_e - q_s)\, s(\tau), \quad s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = t / T \qquad (7)$$

Its peak speed is 1.875 $|\Delta q|/T$ and its peak acceleration 5.77 $|\Delta q|/T^2$. The shortest feasible duration for the limits $\omega_{max} = 6$ rad/s and $\alpha_{max} = 40$ rad/s^2 is therefore:

$$T_{min} = \max_{joints} [\, 1.875\, |\Delta q| / \omega_{max},\ \sqrt{(5.77\, |\Delta q| / \alpha_{max})}\,] \qquad (8)$$

5.4 Joint dynamics and PID control

In the simulator the arm obeys the rigid-body equation $M(q)\, d^2q/dt^2 + C(q, dq/dt) + B\, dq/dt = \tau$ with torque saturation. The controller treats each joint as $J\, d^2q/dt^2 + B\, dq/dt = \tau$ with the constant nominal inertia J_n (taken from the mass matrix at a representative pose, 0.28 and 0.051 kg m^2) and damping B_n, and uses the PID law with feedforward [9], where q_r is the reference trajectory and e the joint error:

$$\tau = J_n\, d^2q_r/dt^2 + B_n\, dq_r/dt + K_p\, e + K_d\, de/dt + K_i \int e\, dt \qquad (9)$$

The gains follow from a desired natural frequency $\omega_n = 50$ rad/s and damping $\zeta = 1$: $K_p = J_n\omega_n^2$, $K_d = 2\zeta\omega_n J_n$, $K_i = 0.2\, K_p\omega_n$. The inertia seen by a joint changes with the arm pose and the two joints are coupled, so the feedforward is imperfect and the PID has to correct the rest.

5.5 Camera model, calibration, colour and shape detection

A planar scene seen by a camera is related to the image by a homography H. With four marker points of known world position the matrix H is estimated by the direct linear transform [11], which OpenCV implements in getPerspectiveTransform [10]:

$$(X, Y, 1)^T \sim H\, (u, v, 1)^T \qquad (10)$$

Colour is classified in HSV space, because hue is almost independent of brightness: red is the hue range below 8 or above 172, green 45 to 85 and blue 95 to 125 (OpenCV scale 0 to 179), each with saturation above 90 and value above 50. Morphological opening removes

isolated noise pixels, and the centroid of each connected component gives the part position with sub-pixel accuracy [19].

The shape is decided from the contour of each component. A circle has the same distance from its centroid to every contour point, a square has distances between half the side and half the diagonal. The coefficient of variation of the contour radius r is therefore a simple, rotation-invariant shape feature:

$$CV = \sigma_r / \mu_r, \quad \text{circle if } CV < 0.06, \text{ square otherwise} \tag{11}$$

At 1.5 mm/pixel the measured values are about 0.02 for the cylinders (pixels spread over 0.019 to 0.026) and 0.09 to 0.11 for the cubes, so the threshold lies far from both classes. At 2.5 mm/pixel the two classes overlapped, which is why the camera resolution was raised.

5.6 Kalman filter for moving objects

The state of each object is $x = [x, y, v_x, v_y]^T$ with a constant-velocity model and white acceleration noise ($\sigma_a = 0.08$ m/s^2) [15], [16]:

$$x_k = F\, x_{k-1} + w, \quad F = [1\ 0\ \Delta t\ 0;\ 0\ 1\ 0\ \Delta t;\ 0\ 0\ 1\ 0;\ 0\ 0\ 0\ 1] \tag{12}$$

$$K = P\,(P + R)^{-1}, \quad x \leftarrow x + K\,(z - x), \quad P \leftarrow (I - K)\,P \tag{13}$$

The measurement z contains the camera position ($\sigma = 3$ mm), the encoder belt speed ($\sigma = 4$ mm/s) and a zero lateral velocity ($\sigma = 2$ mm/s), so that $H = I$. A measurement taken at time t_f reaches the filter at $t_f + 80$ ms. The filter state therefore lags and is extrapolated to the present time: $p(t) = \hat{p} + \hat{v}\,(t - t_f)$. This is the latency compensation.

5.7 Intercept planning

For a tracked object the planner searches the earliest arrival time t_a (in steps of 20 ms) for which the grasp point $p(t_a + T_d)$ lies in the pick window (-0.30 m $\le x \le 0.30$ m), the inverse kinematics has a solution q_g and the minimum-jerk motion from the present pose fits in the available time:

$$t_a - t_{plan} \geq T_{min}(q_{now}, q_g) + 10\ ms \tag{14}$$

Here $T_d = 0.2$ s is the descent time. If no such t_a exists before the object leaves the window, the object is skipped.

6. System Design

6.1 Architecture

Figure 2 shows the architecture. The simulated belt carries the cubes and provides an encoder reading. The camera renders an image 20 times per second; the vision module extracts blobs, which arrive at the tracker 80 ms later. The tracker maintains one Kalman filter per part. When the arm is idle, the planner selects a target and an intercept time, the trajectory generator produces joint references and the PID controllers drive the joints. During the descent the visual-servo block corrects the references with the latest estimate; in CoppeliaSim the joint controller runs in joint-callback scripts inside the physics step. After the grasp the arm carries the part to the bin of its recognised colour (cubes) or to the reject bin (cylinders) and releases it.

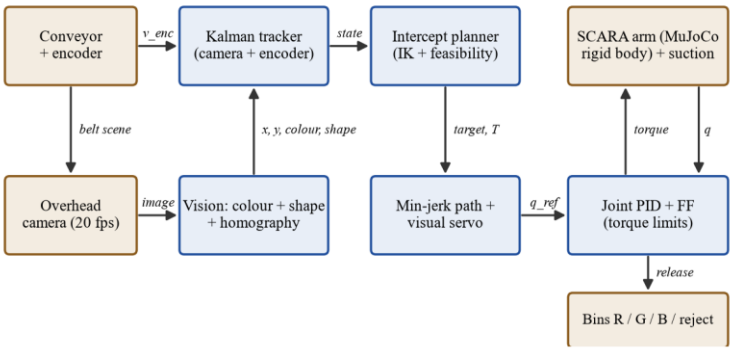


Figure 2: Block diagram of the complete system.

6.2 Robot and environment model

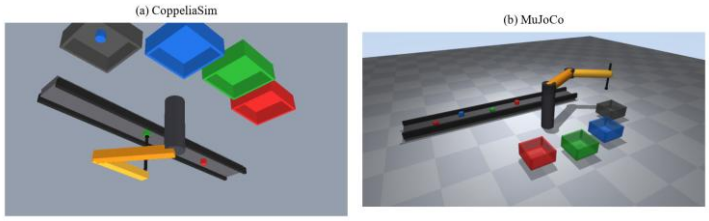


Figure 3: The simulated cell: (a) CoppeliaSim (assigned platform) and (b) MuJoCo; SCARA arm, conveyor, three colour bins and the dark reject bin.

Figure 3 shows the simulated cell. The arm is mounted at the origin of the world frame beside the belt. The belt occupies 0.26 m <= y <= 0.40 m and moves in the +x direction. The camera field of view covers -0.85 m <= x <= 0.35 m and 0.20 m <= y <= 0.46 m, which gives an image of 800 x 173 pixels at 1.5 mm per pixel. Three bins for red, green and blue cubes lie on the opposite side of the arm at (-0.25, -0.30), (0, -0.35) and (0.25, -0.30) m; the reject bin for cylinders is at (0.48, -0.12) m. The principal parameters are given in Table 7.

Table 7: Principal parameters of the simulated cell.

Group	Parameter	Value
Arm	Link lengths L1, L2	0.35 m, 0.30 m
Arm	Joint limits q1, q2	+/- 2.97 rad, +/- 2.53 rad
Arm	Speed / acceleration limit (planner)	6 rad/s, 40 rad/s^2
Arm	Torque limits	45 N m, 18 N m
Arm	Link masses (link 1 / link 2 / tool)	1.8 kg / 1.1 kg / 0.17 kg
Arm	Nominal inertia used by controller	0.28, 0.051 kg m^2
Physics	Engine, time step	CoppeliaSim 4.10.0 (Bullet), 5 ms; MuJoCo 3.17.0, 2 ms
Gripper	Suction capture radius	10 mm
Conveyor	Speed range, ripple, encoder noise	0.05-0.35 m/s, 3 % (0-15 %), 3 mm/s
Objects	Cube side, cylinder diameter, share of cylinders	30 mm, 36 mm, 25 %
Objects	Spacing on the belt	0.35-0.55 m
Camera	Resolution, frame rate, latency	1.5 mm/pixel (800 x 173), 20 fps, 80 ms
Camera	Image noise, calibration noise	8 gray levels, 0.8 mm
Camera	Type	orthographic vision sensor, rendered from the 3-D scene
Control	Joint loop bandwidth, damping	50 rad/s, 1 (both platforms)

6.3 Sensors and actuators

The sensors are the overhead camera (position, colour and shape), the belt encoder (speed) and the joint encoders of the arm (angles and rates, used by the controller and for forward kinematics). The actuators are the two joint motors, the vertical axis and the suction gripper. The camera and encoder deliver data at 20 Hz and the arm controller runs at 500 Hz in MuJoCo (2 ms) and at 200 Hz in CoppeliaSim (5 ms).

6.4 Software architecture

The code is organised in small modules, each of which belongs to one team member (Section 10): arm.py (kinematics, D-H matrices, trajectory, PID law), vision.py (detector, shape classifier, homography), tracker.py (Kalman and line-fit tracking), sim.py (planner and arm state machine), cs_world.py, simcs.py and cs_launch.py (CoppeliaSim scene, closed loop, launcher), mj_world.py and sim3d.py (MuJoCo), the experiment, demo and render scripts, and tests/. The arm state machine has six states: IDLE, MOVE, DESCEND, TRANSPORT, RELEASE and RISE (after a miss).

7. Methodology and Algorithms

7.1 Overall procedure

The complete cycle is summarised in the flowchart of Figure 4. Every 50 ms a frame is rendered and processed; every 20 ms, if the arm is idle, the planner is invoked. Everything else runs at 2 ms.

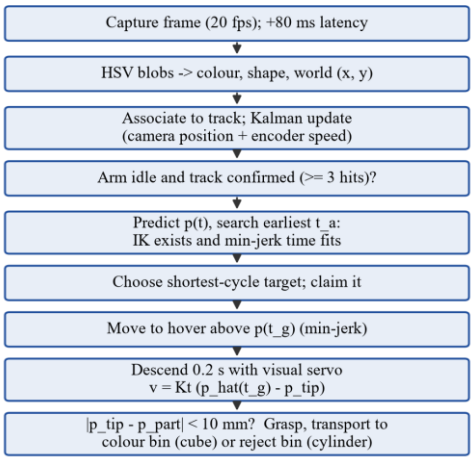


Figure 4: Flowchart of the proposed method (M3), from image capture to bin release.

7.2 Vision pipeline

The camera image is rendered by an orthographic vision sensor above the belt (CoppeliaSim; MuJoCo uses an equivalent camera), showing parts, belt and the arm itself, which can occlude parts. The brightness is scaled by a time-varying lighting factor (+/- 12 %) and Gaussian pixel noise (default 8 gray levels) is added. The detector blurs the image, converts it to HSV, applies the three colour thresholds, removes noise by opening, labels connected components and discards blobs that are too small or touch the image border (partly visible parts would bias the centroid). For every remaining blob the contour is extracted and the shape feature CV of Section 5.5 gives the class square or circle. Figure 5 shows an example frame and the detections. Pixel centroids are mapped to world coordinates with the homography of Section 5.5.

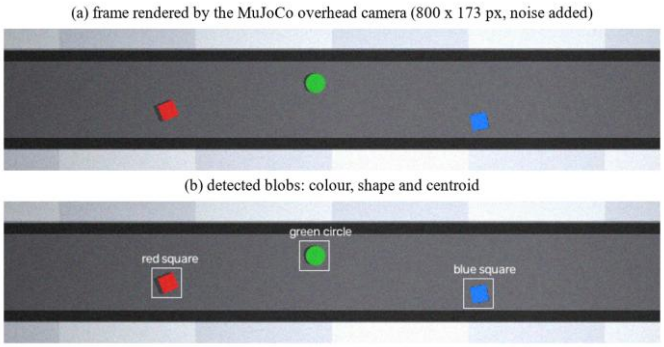


Figure 5: (a) Camera frame rendered by MuJoCo and (b) detected blobs with colour, shape and centroid.

7.3 Tracking and prediction

Detections are assigned to existing tracks by nearest neighbour within a 50 mm gate; unassigned detections start new tracks and tracks that are not updated for 0.6 s are deleted. A track is confirmed after three detections and the class label is decided by majority vote. Two estimators were implemented. The baseline fits a straight line to the last twelve measurements by least squares. The proposed estimator is the Kalman filter of Section 5.6 with encoder fusion and latency compensation.

7.4 Planning algorithm

Algorithm 1 gives the planner in pseudo-code.

```
Algorithm 1: Intercept planning (called when the arm is idle)
1  for each confirmed, unclaimed track tr with fails < 2:
2      (x, y, vx, vy) <- tr.estimate(t_now)      # latency compensated
3      if x outside [-0.30, 0.30]: continue
4      for t_a = t_now + 0.10 ... in steps of 0.02 s:
5          g <- (x, y) + (vx, vy) * (t_a + T_d - t_now)   # grasp point
6          if g.x > 0.30: break                  # object would be lost
7          q_g <- IK(g) nearest to q_now; if none: continue
8          if t_a - t_now - t_plan >= T_min(q_now, q_g) + 0.01:
9              add (tr, t_a, g, q_g) to candidates; break
10 pick the candidate with the shortest cycle time (FIFO: the first)
11 claim it; plan minimum-jerk motion q_now -> q_g, arrival at t_a
```

7.5 Predictive visual-servo descent

After arrival at the hover height the tip descends 0.10 m in $T_d = 0.2$ s. In the plain predictive method the arm holds the planned point during the descent. In the proposed method the references are updated every 2 ms. The latest Kalman estimate is extrapolated to the grasp time t_g and the tip is driven towards it with a proportional law in Cartesian space and the inverse Jacobian (Algorithm 2, $K_t = 15\ s^{-1}$). The point $\hat{p}(t_g)$ is used rather than the current object position, because the arm arrives ahead of the cube and waits for it, as shown in the traces of Section 9.

```
Algorithm 2: Visual-servo descent (every 2 ms until t_g = t_arrival + T_d)
1  z <- minimum-jerk descent from z_hover to z_pick
2  p_goal <- tr.estimate(t_g)        # predicted cube position at contact
3  p_tip  <- FK(q_measured)
4  v_cmd  <- Kt * (p_goal - p_tip)
5  dq     <- J(q_ref)^-1 * v_cmd
6  q_ref  <- q_ref + dq * dt ;  qdot_ref <- dq
7  at t_g: grasp succeeds if |p_tip - p_cube| < r_cap = 10 mm
```

7.6 Controller design and tuning

The gains were chosen from the pole-placement formulas of Section 5.4 with $\omega_n = 50$ rad/s (about eight times faster than the minimum-jerk motions). The gains were verified with a 0.5 rad step and a 1.2 rad minimum-jerk tracking test (Section 9.1). With a 2 ms step and semi-implicit Euler integration the loop is stable ($\omega_n\, dt = 0.1$). The integral term is clamped to avoid wind-up when the torque saturates.

7.7 Methods compared

Table 8 lists the three strategies and the three variants that are used in the ablation study.

Table 8: The three strategies and the ablated variants.

Method	Belt	Estimator	Descent	Scheduling
M1 Stop-and-go	stops while an object is in the window	mean of still-belt detections	hold	nearest first
M2 Predictive	moves	least-squares line fit	hold planned point	FIFO
M3 Proposed	moves	Kalman + encoder	visual servo	shortest cycle (SCT)
M3 w/o visual servo	moves	Kalman + encoder	hold	SCT
M3 w/o Kalman	moves	line fit	visual servo	SCT
M3 w/o SCT	moves	Kalman + encoder	visual servo	FIFO

8. Simulation Setup

8.1 Platform

The assigned platform of Group G10 is CoppeliaSim [4] (vision sensor, arm and conveyor) with Python and OpenCV. The cell is built in CoppeliaSim V4.10.0 (EDU) with the Bullet physics engine and is controlled from Python through the ZMQ remote API. The software of the project (detector, tracker, planner, visual servo) is plain Python 3 with NumPy, OpenCV [10] and SciPy and is identical on both platforms. CoppeliaSim hosts the live demonstration, the video and a cross-platform validation study. One CoppeliaSim trial of 12 parts takes about 40 to 80 s of computing time, about five times longer than in MuJoCo [5], so the large statistical studies were run in MuJoCo. Running the identical software on

two independent physics engines also shows that the results are not an artefact of one engine (Section 9.7).

In CoppeliaSim the scene is created from Python with the remote API (cs_world.py): static shapes for the belt and the four bins, two dynamic revolute joints with their links, a kinematic vertical axis with the suction tool, an orthographic vision sensor, and parts created at the start of the belt. The joint-level PID with feedforward of Section 5.4 is evaluated in a joint-callback script inside every physics step (exact timing), and one batched call per step sets the references and moves the conveyor parts. In MuJoCo the arm, the belt and the bins are defined in an MJCF model (mj_world.py) and the torque controller runs in Python. Randomness (part colour, shape, position, orientation, spacing, image noise and calibration error) is controlled by an integer seed so that the same seeds are used for all methods. Table 9 maps the elements named in the Project List to the implementation.

Table 9: Elements of the CoppeliaSim set-up named in the Project List and their implementation.

Element (Project List)	Implementation in this project
Vision sensor	Orthographic vision sensor, 800 x 173 px, 20 fps; the image goes to OpenCV, noise and latency are added in Python
Arm	SCARA: base, two dynamic revolute joints, links, kinematic vertical axis, suction tool; D-H table of Table 6; PID + feedforward in a joint callback
Conveyor	Belt shape; the parts are moved along it every step with speed ripple (kinematic); the encoder value goes to the tracker
Parts and bins	Cubes and cylinders as Bullet rigid bodies; four open bins; success read from the physics state
Python + OpenCV	Detector, tracker, planner and visual servo: the same code on CoppeliaSim and MuJoCo

8.2 Experiments

- **MuJoCo belt speed study:** 3 methods x 7 speeds (0.05 to 0.35 m/s) x 10 seeds, 15 parts per run (150 parts per point).
- **MuJoCo stress tests at 0.25 m/s (M2 and M3):** latency 0 to 320 ms, speed ripple 0 to 15 %, image noise 4 to 60 gray levels; ablation at 0.30 m/s with 6 % ripple and 160 ms latency (five variants).
- **MuJoCo confirmation study:** 30 new seeds for the key comparisons (330 runs, Section 9.6).
- **CoppeliaSim cross-platform study:** 105 runs with 5 seeds and 12 parts per run on 8 headless CoppeliaSim instances: 3 methods at 4 speeds, stress tests at 15 % ripple and 320 ms latency, and the ablation (Section 9.7).

- **Controller and vision studies:** joint step response and minimum-jerk tracking on both platforms, and a detection study with 270 parts per condition.

8.3 Performance measures

A part counts as sorted correctly only if, after the simulation, it physically rests inside its correct bin (cubes: the bin of their true colour; cylinders: the reject bin) (the position is read from the physics engine, not from the planner). The success rate is the percentage of all parts that were sorted correctly; parts that left the pick window are counted as lost. The throughput is the number of correctly sorted parts per minute of simulated time, measured until the last part is resolved. The grasp error is the tip-to-part distance at the moment of suction.

8.4 How to run the code

Table 10 lists the commands that reproduce every result of this report.

Table 10: Files of the repository and commands to reproduce all results.

Command	Purpose
pip install -r requirements.txt	install the libraries (CoppeliaSim: docs/COPPELIASIM_SETUP.md)
python demo_cs.py [--method m2 --speed 0.3]	live demo in the CoppeliaSim window (used in the video)
python render_cs.py --stills	CoppeliaSim stills and MP4 video
python demo.py / python render_3d.py	the same demo and renders in MuJoCo
python tests/test_core.py	11 unit tests
python experiments.py [--confirm]	560 + 330 MuJoCo runs -> results/*.csv (about 25 min)
python experiments_cs.py	105 CoppeliaSim runs -> results/cs_runs.csv (about 15 min)
python extra_experiments.py / python extra_cs.py	controller and vision studies on both platforms
python make_figures.py, python report/build_report.py	regenerate figures, rebuild the report

8.5 Model validation and tests

Eleven unit tests (tests/test_core.py) check the building blocks, and the checks of Table 11 show that the planner, the D-H model and both physics models use the same geometry and that the detector, the tracker and the minimum-jerk planner behave as specified.

Table 11: Validation checks of the simulation.

Check	Result
IK followed by FK on 500 random targets	max error < 1e-9 m
Analytic vs finite-difference Jacobian	max difference < 1e-5
MuJoCo tool position vs analytic FK (50 poses)	max error < 1e-9 m
Minimum-jerk peak speed factor	1.875 (exact)
Kalman velocity estimate, 0.2 m/s belt	error < 0.01 m/s
D-H chain A1 A2 A3 vs closed-form tip position (100 poses)	max error < 1e-12 m
CoppeliaSim tool tip vs analytic FK (q1 = 0.8, q2 = 1.0)	(0.176, 0.543) m in both
CoppeliaSim vision sensor vs true part positions (4 parts)	centroid error <= 1.5 mm, colour and shape all correct
Detector on a rendered scene (squares, circles, 3 colours)	colour and shape all correct
Parts counted as sorted	verified from physics state, not from planner log

9. Results and Analysis

9.1 Joint controller

Figure 6(a) shows the response of joint 1 of the MuJoCo arm to a 0.5 rad step. A proportional controller alone oscillates (78 % overshoot, not settled after 0.6 s). PD control has 0.0 % overshoot and settles within 2 % in 0.14 s; PID adds an integral term with 5.4 % overshoot and 0.26 s settling, which removes the steady-state error that joint damping and coupling torques would otherwise leave while the arm holds the hover pose. For the minimum-jerk motions used in the cell, panel (b), the maximum tracking error is 5.6 mrad with PD, 7.9 mrad with PID and 5.7 mrad with PID plus feedforward. The feedforward cuts the PID error by 28 %; its peak error equals that of PD, because the nominal inertia is only an approximation of the pose-dependent inertia of the coupled arm. At the 0.4 m reach, 5.7 mrad corresponds to 2.3 mm at the tip.

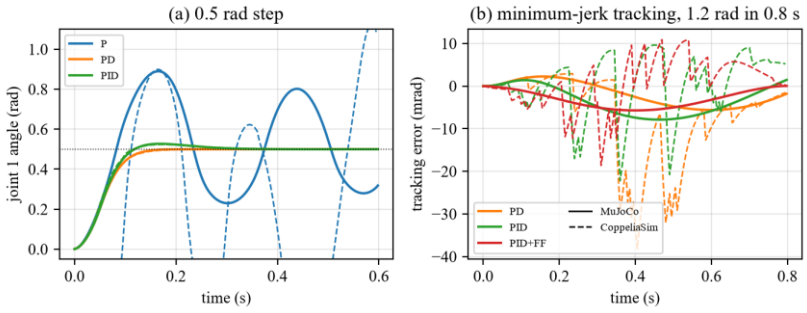


Figure 6: Joint 1 controller: (a) step response; (b) tracking error (solid: MuJoCo, dashed: CoppeliaSim).

The same tests in CoppeliaSim (dashed curves; the PID + feedforward law runs in a joint-callback script every 5 ms) give the same qualitative result: the P-only loop oscillates (125

% overshoot), PD has 0.0 % overshoot and PID 4.7 %. The peak tracking error of the minimum-jerk motion is 38.2 mrad with PD, 22.2 mrad with PID and 18.6 mrad with PID plus feedforward, about three times larger than in MuJoCo because the loop is sampled at 200 Hz instead of 500 Hz and the Bullet joints are softer. This corresponds to 7.4 mm at the 0.4 m reach and explains the larger grasp error of the CoppeliaSim cell.

9.2 Vision module

The detector was tested on 270 parts per condition for image noise from 4 to 80 gray levels and for three lighting levels (Figure 7). Colour classification is the more robust feature: it is 100 % correct at every noise level up to 60 and 99 % at noise 80 with light x0.7. Shape classification is the limiting step: it is 100 % correct at noise 25 and 98.9 % at noise 40 with nominal light, and falls to 87 % at noise 60 and 69 % at noise 60 with light x0.7, because noise makes the contour of a cylinder ragged and raises its radius variation CV towards the threshold. A 5 x 5 Gaussian pre-blur limits this effect; the default noise (8 gray levels) is far below these limits. The position RMSE is about 2.2 mm. Without calibration noise it is 1.4 mm (the detector itself: 1.5 mm pixels, shadows and the part edges); the 0.8 mm noise on the four calibration markers raises it to 2.2 mm, so the calibration is the larger part of the perception error; image noise adds almost nothing. Missed detections are bridged by the tracker, which also decides colour and shape by majority vote over all frames.

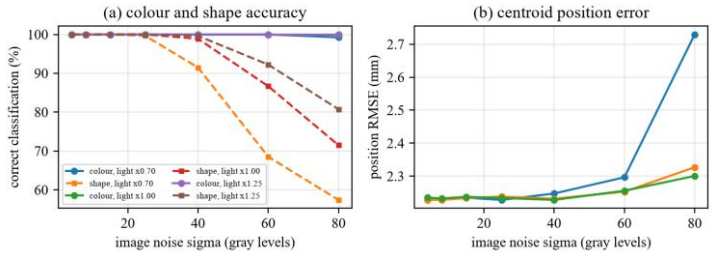


Figure 7: Vision accuracy versus image noise for three lighting levels.

9.3 Comparison of the three strategies

Table 12 and Figure 8 compare the strategies over belt speed (mean of 10 seeds, error bars: one standard deviation). The stop-and-go cell (M1) sorts 100 % of the cubes at every speed, but its throughput is only 5.1 to 14.6 objects per minute, because the whole belt waits for each pick. The moving-belt methods are much faster: at 0.25 m/s the proposed method reaches 28.5 objects per minute, 2.2 times the stop-and-go rate (objective O4). Up to 0.10 m/s M2 and M3 sort at least 99 % of the cubes. From 0.15 m/s on M2 starts to fail (97.3 % at 0.15 m/s, 90.7 % at 0.20 m/s) because its aim point is off by 4 to 5 mm and the

suction cup captures only 10 mm, so it makes 2.1 failed grasp attempts per run at 0.25 m/s, while M3 makes 0.0. M3 sorts 100.0 % at 0.20 m/s (objective O3). The mean grasp error of M2 is 2.8 to 5.2 mm, that of M3 2.7 to 3.4 mm (objective O2). Even the stop-and-go cell has 2.6 mm; this is the perception floor (calibration and centroid error, Section 9.2), so no method can be much better. Above 0.25 m/s the success of both moving-belt methods drops (M3: 78.0 % at 0.30 m/s and 67.3 % at 0.35 m/s) for a different reason than accuracy: the arm needs about 2 s for a full pick-and-place cycle, so objects arrive faster than they can be served and some leave the pick window. This is a capacity limit of the single arm; M3 still makes no failed grasps.

Table 12: Results versus belt speed (mean of 10 runs with 15 objects each).

Speed (m/s)	M1 succ. %	M1 obj/min	M2 succ. %	M2 err mm	M3 succ. %	M3 obj/min	M3 err mm
0.05	100.0	5.1	100.0	2.8	100.0	6.5	2.7
0.10	100.0	8.4	100.0	3.5	100.0	12.7	3.1
0.15	100.0	10.5	97.3	4.3	100.0	19.5	3.3
0.20	100.0	12.1	90.7	4.4	100.0	24.7	3.4
0.25	100.0	13.2	86.7	4.7	93.3	28.5	3.3
0.30	100.0	14.0	68.7	4.7	78.0	27.7	3.2
0.35	100.0	14.6	60.0	5.2	67.3	28.1	3.2

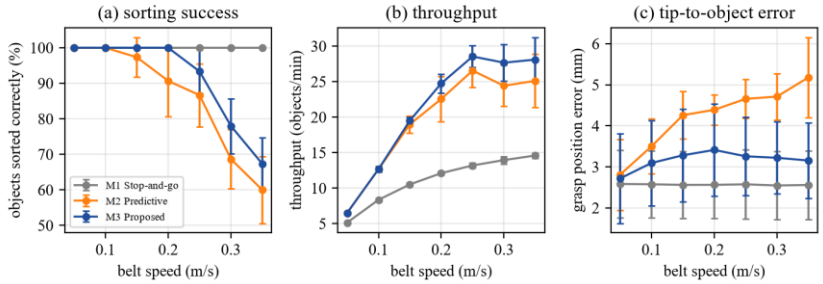


Figure 8: (a) Sorting success, (b) throughput and (c) grasp error versus belt speed.

Inspection quality. In all runs with image noise up to 40 (the 560-run study and the 330-run confirmation study together: 3213 cylinders and 9837 cubes) 1 defective part was accepted into a colour bin (false accept, 0.03 %) and 0 good cubes were sent to the reject bin; no cube was delivered to a wrong colour bin (objective O5). The single false accept happened with the plain predictive method M2 at 15 % belt-speed ripple: the track followed a blue cube but the suction cup reached a neighbouring blue cylinder, so the bin was chosen from the wrong track (verified by replaying the run). All other sorting failures are parts that were not handled in time (lost from the belt), never a wrong classification. Only in the extreme-noise runs (noise 60, 300 parts, 70 cylinders) did the shape classifier

fail: 22 cylinders were accepted into a colour bin. Colour classification was correct in all runs.

9.4 Robustness to latency, speed ripple and image noise

Figure 9 shows the stress tests at 0.25 m/s. When the vision latency is increased from 0 to 320 ms, the success of M2 falls from 90.0 % to 76.7 % and its grasp error rises from 4.2 to 4.8 mm; M3 keeps its success at 93.3 % at every latency and its error between 3.0 and 3.3 mm, because the latency is compensated in the filter and the descent is corrected with the latest estimate. The speed ripple has the strongest effect: at 15 % ripple M2 sorts only 48.7 % of the cubes, with a grasp error of 5.6 mm, while M3 still sorts 88.0 % with 4.0 mm (objective O3: 5.3 points lower than at 3 % ripple). A speed error of 1 % over a 1 s look-ahead at 0.25 m/s shifts the target by 2.5 mm, so a ripple of this size is critical for a 10 mm suction cup. Image noise up to 40 gray levels changes neither method beyond the run-to-run scatter (M3: 93.3 % at every level; M2: 86.0 to 86.7 %); at noise 60 the success falls to 84.0 % (M3) and 76.0 % (M2) because of the shape errors described in Section 9.2.

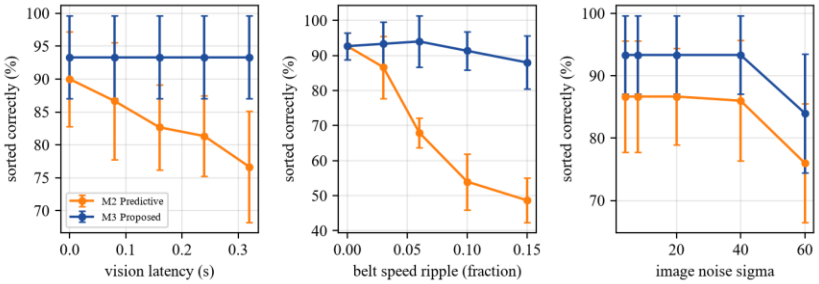


Figure 9: Success at 0.25 m/s versus latency, ripple and image noise.

9.5 What happens during one grasp

Figure 10 compares the same 8-object trial for M2 and M3 at 0.25 m/s. Panel (a) shows the tip-to-object distance in the 0.2 s before contact. Both methods start about 50 mm apart, because the arm waits ahead of the cube; M3 converges smoothly to a few millimetres while the final distance of M2 depends on the accumulated prediction error. Panel (b) shows the error at contact for each pick: M2 varied between 0.9 and 13.5 mm and 2 of its 9 contacts were outside the 10 mm capture radius (misses that needed a second attempt), whereas M3 stayed between 2.7 and 4.1 mm with 0 misses. Figure 11 shows three moments of an M3 trial rendered from the 3-D scene.

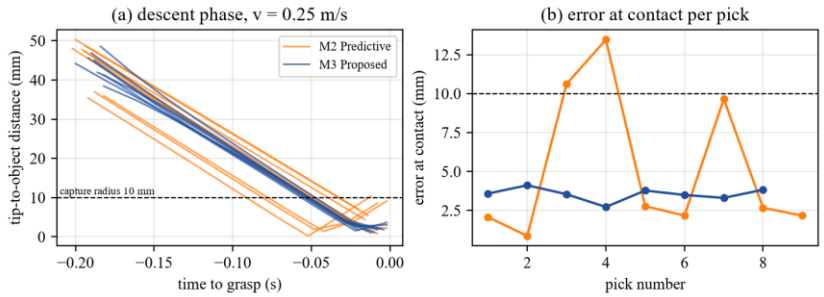


Figure 10: Same trial for M2 and M3: approach and error at contact.

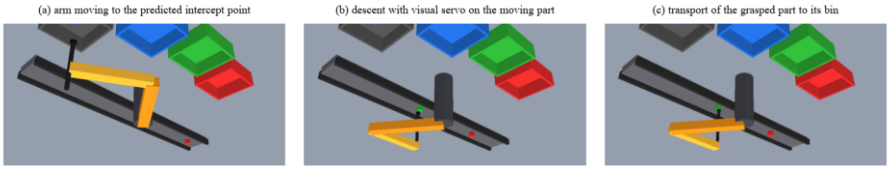


Figure 11: Snapshots of one trial of the proposed method.

9.6 Ablation study

Table 13 and Figure 12 isolate the components in a hard condition (0.30 m/s, 6 % ripple, 160 ms latency). Starting from the plain predictive method M2 (54.0 %), adding the Kalman filter with encoder fusion and the visual-servo descent together gives the full M3 (78.0 %). Removing the visual servo from M3 costs 14.0 points of success and raises the grasp error from 3.3 mm to 4.7 mm; removing the Kalman filter costs 6.0 points and raises the error to 4.5 mm. Both components therefore contribute, and the visual servo contributes more. The scheduling rule makes no difference (78.0 % with and without). With 10 seeds the standard deviations are 6 to 12 points, so the Kalman difference (Welch t-test $p = 0.20$) could not be separated from the run-to-run scatter. The confirmation study below, with 30 new seeds, removes this doubt. The ordering M2 < no servo < no Kalman < full M3 is clear, and full M3 also has the lowest grasp error of all variants (3.32 mm).

Table 13: Ablation at 0.30 m/s, 6 % ripple, 160 ms latency (10 runs, 15 objects each).

Variant	Success %	Std (pts)	Throughput obj/min	Grasp error mm
M2 Predictive	54.0	8.6	20.0	4.96
M3 without visual servo	64.0	12.3	23.7	4.73
M3 without Kalman (line fit)	72.0	11.7	26.6	4.50
M3 without SCT (FIFO)	78.0	8.3	28.6	3.32
M3 Proposed (full)	78.0	8.3	28.6	3.32

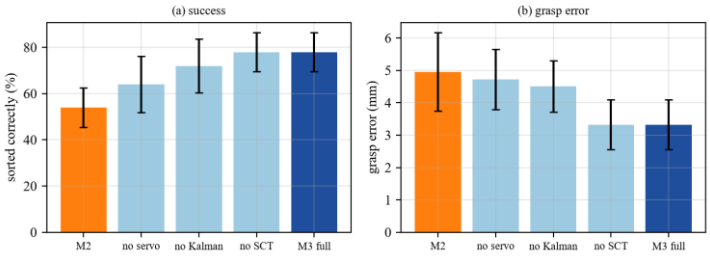


Figure 12: Ablation: (a) sorting success and (b) grasp error of the variants.

Statistical confirmation. To make the claims independent of the first 10 seeds, the key comparisons were repeated with 30 new seeds (330 runs, 15 parts each; Table 14). Every component of the proposed method gives a significant gain (Welch t-test of each variant against the full method): removing the Kalman filter costs 8.2 points ($p < 0.001$), removing the visual servo costs 11.1 points ($p < 0.001$) and the plain predictive method M2 is 23.1 points lower ($p < 0.001$). The scheduling rule remains without effect (identical results). Table 15 compares M3 with M2 under other conditions: the advantage is significant at 0.25 m/s and 0.30 m/s and very large at 15 % belt-speed ripple.

Table 14: Confirmation study of the ablation with 30 new seeds (450 parts per variant).

Variant (0.30 m/s, 6 % ripple, 160 ms)	Success %	Std (pts)	Grasp error mm	p vs full M3
M2 Predictive	54.7	6.2	4.60	< 0.001
M3 without visual servo	66.7	8.9	4.87	< 0.001
M3 without Kalman (line fit)	69.6	7.8	4.39	< 0.001
M3 without SCT (FIFO)	77.8	6.1	3.32	1.000
M3 Proposed (full)	77.8	6.1	3.32	-

Table 15: M3 versus M2 in the confirmation study (30 new seeds).

Condition (30 seeds)	M3 success %	M2 success %	Difference (pts)	p
0.25 m/s	93.3	87.6	5.8	0.009
0.30 m/s	77.1	69.6	7.6	< 0.001
0.25 m/s, 15 % ripple	92.9	47.1	45.8	< 0.001

9.7 Cross-platform validation in CoppeliaSim

The complete software was also run in CoppeliaSim, the assigned platform, with 105 runs of 12 parts each (1260 parts, of which 318 cylinders). Table 16 compares it with the MuJoCo study. A part counts only if it rests in its correct bin in the CoppeliaSim physics; in 105 of 105 runs this check agreed with the planner log. The ranking of the methods is the same on both engines: M1 (stop-and-go) sorts everything but slowly, and M3 is at least as successful as M2 in 4 of 4 speeds, with 78.3 % against 76.7 % at 0.30 m/s. The grasp error of M3 is 2.9 to 3.7 mm in CoppeliaSim and 2.7 to 3.4 mm in MuJoCo; it is larger in

CoppeliaSim because the joint loop is evaluated every 5 ms and the Bullet contacts are softer. Inspection in CoppeliaSim: 0 false accepts and 0 false rejects among 318 cylinders and 942 cubes.

Table 16: Speed study in CoppeliaSim (5 seeds, 12 parts) and in MuJoCo (10 seeds, 15 parts).

Method	v (m/s)	CoppeliaSim succ. %	MuJoCo succ. %	CoppeliaSim obj/min	MuJoCo obj/min	CS err mm	MuJoCo err mm
M1	0.10	100.0	100.0	8.3	8.4	2.1	2.6
M1	0.20	100.0	100.0	11.9	12.1	2.1	2.6
M1	0.25	100.0	100.0	13.1	13.2	2.1	2.6
M1	0.30	100.0	100.0	13.7	14.0	2.1	2.5
M2	0.10	100.0	100.0	12.6	12.7	3.5	3.5
M2	0.20	98.3	90.7	24.0	22.6	4.2	4.4
M2	0.25	85.0	86.7	25.4	26.6	4.3	4.7
M2	0.30	76.7	68.7	26.7	24.4	5.2	4.7
M3	0.10	100.0	100.0	12.6	12.7	2.9	3.1
M3	0.20	100.0	100.0	24.3	24.7	3.4	3.4
M3	0.25	95.0	93.3	28.4	28.5	3.2	3.3
M3	0.30	78.3	78.0	27.5	27.7	3.7	3.2

The stress tests and the ablation give the same picture on both engines. At 15 % belt-speed ripple M3 sorts 88.3 % (CoppeliaSim) and 88.0 % (MuJoCo) of the parts, M2 only 46.7 % and 48.7 %. With 320 ms vision latency M3 reaches 95.0 % and 93.3 %, M2 80.0 % and 76.7 %. In the ablation at 0.30 m/s the success rises step by step from M2 (56.7 % / 54.0 %) through M3 without visual servo (66.7 % / 64.0 %) and M3 without Kalman filter (73.3 % / 72.0 %) to the full method M3 (83.3 % / 78.0 %). With only 5 seeds per point the CoppeliaSim differences are less precise than the MuJoCo results, which is why the statistics of Section 9.6 were done in MuJoCo.

9.8 Discussion: what worked and what did not

- **Worked:** predicting the intercept, correcting the grasp point during the descent and fusing the encoder. These keep the success high (M3: 93.3 % at 0.25 m/s) although the camera is 80 ms late, the belt speed fluctuates and the arm hides parts of the scene.
- **Worked:** the minimum-jerk plus feedforward motion: it is smooth, respects the limits and is tracked within 5.7 mrad.
- **Did not work:** the SCT scheduling rule (no gain) and chasing the current object position during the descent (our first version, which failed completely above 0.25 m/s because the arm arrives ahead of the cube by about 50 mm).
- **Limit:** above about 0.25 m/s the single arm is saturated at roughly 29 objects per minute; a second arm or faster motion limits would be needed. The camera is

orthographic, the conveyor is kinematic and the suction is an attach constraint, so absolute numbers will differ on a real robot.

10. Contribution

Each member owns one technical module. All claims below can be traced to the files of the GitHub repository (Appendix) and to the corresponding part of the video. The member who built a module presents it in the video and slides. Table 17 gives the contribution of each member.

Table 17: Technical contribution of each member and report sections written.

S. No.	Member name and SAP ID	Technical contribution (modules / algorithms / code files / experiments)	Report sections written + signature
1	Aditya Khatri (500119453)	Built the SCARA kinematics in arm.py (forward and inverse kinematics with joint limits and elbow choice, Jacobian, D-H matrices, minimum-jerk generator and the duration formula) and the arm state machine and the visual-servo descent in sim.py (_arm, _grasp, _release). Also built the CoppeliaSim cell (cs_world.py: scene, arm, vision sensor, bins, suction attach; simcs.py: closed loop) and the MuJoCo cell in mj_world.py (arm, belt, bins, free-body cubes, orthographic camera, suction weld) and the closed loop in sim3d.py. Produced Fig. 1, 3, 10 and 11.	Sections Abstract, 1, 2, 12, Appendix Sign: _____
2	Simran Jeet Gill (500122195)	Built the camera and detector in vision.py (rendering of rotated cubes and cylinders, noise and lighting, HSV thresholds, morphological filtering, connected components, contour shape classifier, homography calibration) and the vision study (extra_experiments.py: vision_study). Produced Fig. 5 and 7.	Sections 3, 4, 8 Sign: _____
3	Divya Israni (500121919)	Built the tracker in tracker.py (nearest-neighbour association, Kalman filter with encoder pseudo-measurement and latency extrapolation, line-fit baseline) and the intercept planner in sim.py (_intercept, _plan, SCT/FIFO scheduling). Ran the latency and ablation studies. Produced Fig. 4 and 12.	Sections 5, 6 Sign: _____
4	Amritanshi Singh (500119477)	Built the joint plant and PID controller with feedforward in arm.py (JointPlant), derived the gains, ran the step-response and tracking tests (extra_experiments.py: step_response, traj_tracking) and tuned wn, zeta and the integrator clamp. Wrote the torque-level controller JointPlantMJ in sim3d.py and the joint-callback PID + feedforward script of the CoppeliaSim cell (cs_world.py), tested in extra_cs.py. Produced Fig. 6.	Sections 7, 9.1-9.2, References Sign: _____
5	Anchit Saxena (500122868)	Built the experiment harness (experiments.py, parallel runs and CSV logging), the stop-and-go baseline logic in sim.py (belt stop/go and static position estimate), make_figures.py, experiments_cs.py (8 parallel CoppeliaSim instances), cs_launch.py, demo_cs.py and render_cs.py (live CoppeliaSim demo, stills, video), demo.py and render_3d.py (MuJoCo), the unit tests and the README. Produced Fig. 2, 8, 9 and the result tables.	Sections 9.3-9.8 Sign: _____

11. Co-learning Reflection

[Each member must replace the draft below with their own words before printing.]

Aditya Khatri: On my own I learned how inverse kinematics, the Jacobian and D-H matrices turn into code, and why the elbow solution closest to the present pose avoids joint

flips. Debugging my first visual servo showed me that the arm arrives ahead of a moving part and must aim at the predicted contact point. From my team-mates I learned how the Kalman filter and the PID gains shape my trajectories. I taught the team how minimum-jerk durations follow from speed and acceleration limits.

Simran Jeet Gill: I learned HSV colour segmentation and homography calibration from the OpenCV documentation and by testing thresholds on rendered images, and I designed the contour feature that tells cylinders from cubes. I saw that shape fails before colour when image noise grows. From my team-mates I learned how detections feed the tracker and why latency matters. I taught the others how pixels become world coordinates.

Divya Israni: I studied the Kalman filter from the tracking literature and implemented a constant-velocity model with an encoder pseudo-measurement. I learned that a filter only helps when latency is handled, which led to the extrapolation step. From my team-mates I learned the controller and vision details that feed my tracker. I taught the others how to read the ablation tables and why the scheduling rule gave no benefit.

Amritanshi Singh: I learned to design a PID controller from a desired natural frequency and damping and to add model feedforward. Testing on the arm showed me that a P-only loop oscillates, that PD is smooth, and that the pose-dependent inertia of a coupled arm limits what feedforward can do. From my team-mates I learned how my controller is used in the visual-servo descent. I taught the team how to read the step and tracking plots.

Anchit Saxena: I learned to compare methods fairly: same seeds, many runs, error bars and significance tests. Counting success from the physics state instead of the planner log showed me that parts pile up in a bin, which we fixed with a release grid. From my team-mates I learned the algorithms I evaluated. I taught the others how to run the scripts, rebuild the figures and start the live demo.

12. Conclusion and Future Scope

A complete vision-guided colour-and-shape sorting cell for quality inspection was designed and simulated in CoppeliaSim (the assigned platform) and in the MuJoCo physics engine: an OpenCV detector on rendered camera images, a latency-compensated Kalman tracker with encoder fusion, an intercept planner, minimum-jerk motion, joint PID with feedforward and a predictive visual-servo descent. Over 560 MuJoCo runs, in which a part counts only if it physically rests in the right bin, the proposed method sorted 100.0 % of the cubes at 0.20 m/s and 93.3 % at 0.25 m/s, kept the mean grasp error between 2.7 and 3.4 mm (the perception floor of this camera is about 2.6 mm), stayed robust at 15 % belt-

speed ripple (88.0 % success against 48.7 % for the plain predictive method) and delivered 2.2 times the throughput of a stop-and-go conveyor at 0.25 m/s. Inspection errors were extremely rare: one false accept in more than 3000 cylinders and no false reject at image noise up to 40. The same software reproduced the ranking of the methods in 105 CoppeliaSim runs (Section 9.7). The ablation showed that the visual-servo descent matters most, the Kalman filter helps significantly too (30-seed confirmation study, Section 9.6), and the scheduling rule did not help.

Industrial use. The same structure applies to quality inspection and sorting in manufacturing (the application domain of this project), parcel and component sorting, food grading and recycling lines. The study also gives a design rule: the capture radius of the gripper and the ripple of the belt decide whether open-loop prediction is enough or whether a feedback correction in the descent is needed.

Transfer to a real robot. The modules map directly onto hardware: a real camera with a hardware trigger replaces the renderer, the encoder feeds the filter, the planner and the visual servo run on the robot controller and the joint PID is replaced by the drive's servo loop. Remaining work includes hand-eye calibration, lens distortion, gripper vacuum delay and collision checking. Future extensions are a second arm to raise the capacity, learning-based detection for unknown objects, orientation-aware grasping with a gripper, an unscented filter for varying belt speed, a ROS integration and a hardware-in-the-loop test on a real SCARA arm.

References

[1] R. Clavel, "Delta, a fast robot with parallel geometry," in *Proc. 18th Int. Symp. Ind. Robots (ISIR)*, Sydney, Australia, Apr. 1988, pp. 91-100.

[2] FANUC Corporation, "iRVision integrated robot vision and line tracking," product documentation, Oshino, Japan. [Online]. Available: https://www.fanuc.eu. (accessed: 27 Oct. 2026)

[3] ABB Robotics, "PickMaster conveyor-tracking software for FlexPicker robots," product information, Zürich, Switzerland. [Online]. Available: https://new.abb.com/products/robotics. (accessed: 27 Oct. 2026)

[4] E. Rohmer, S. P. N. Singh, and M. Freese, "CoppeliaSim (formerly V-REP): A versatile and scalable robot simulation framework," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Tokyo, Japan, Nov. 2013, pp. 1321-1326.

[5] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Vilamoura, Portugal, Oct. 2012, pp. 5026-5033.

[6] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.

[7] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. New York, NY, USA: Pearson, 2018.

[8] T. Flash and N. Hogan, "The coordination of arm movements: An experimentally confirmed mathematical model," *J. Neurosci.*, vol. 5, no. 7, pp. 1688-1703, Jul. 1985.

[9] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: ISA, 1995.

[10] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, vol. 25, no. 11, pp. 120-125, Nov. 2000.

[11] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[12] F. Chaumette and S. Hutchinson, "Visual servo control, Part I: Basic approaches," *IEEE Robot. Autom. Mag.*, vol. 13, no. 4, pp. 82-90, Dec. 2006.

[13] F. Chaumette and S. Hutchinson, "Visual servo control, Part II: Advanced approaches," *IEEE Robot. Autom. Mag.*, vol. 14, no. 1, pp. 109-118, Mar. 2007.

[14] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Cham, Switzerland: Springer, 2017.

[15] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35-45, Mar. 1960.

[16] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.

[17] *Manipulating Industrial Robots - Performance Criteria and Related Test Methods*, ISO Standard 9283, 1998.

[18] J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," *J. Appl. Mech.*, vol. 22, no. 2, pp. 215-221, Jun. 1955.

[19] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

Appendix: Links and Logs

A.1 Project links

GitHub repository: https://github.com/Aditya-Khatri-7/Robotics-assignment-project-b4

YouTube video (unlisted): https://youtu.be/<unlisted-video-id>

A.2 Meeting log

Table 18 records the meetings of the group.

Table 18: Meeting log of the group.

No.	Date	Duration	Attendees	Decisions
1	__/__/2026	__ min	All 5	Group formed, lead chosen, modules divided, simulator decided (CoppeliaSim + Python/OpenCV)
2	__/__/2026	__ min	All 5	Architecture and parameters agreed; GitHub repository created
3	__/__/2026	__ min	All 5	Results discussed; novelty points fixed; report writing split
4	__/__/2026	__ min	All 5	Report and slides reviewed; video rehearsal

A.3 Peer-assessment sheets

One sheet per member (Table 19): each member scores every team-mate from 1 to 5 and signs the own sheet in the printed copy.

Table 19: Peer-assessment sheets (one per member).

No.	Team-mate name and SAP ID	Contribution (1-5)	Reliability / on time (1-5)	Helped others learn (1-5)	Quality of work (1-5)
	Sheet 1, filled by: Aditya Khatri (500119453)				
1	Simran Jeet Gill (500122195)				
2	Divya Israni (500121919)				
3	Amritanshi Singh (500119477)				
4	Anchit Saxena (500122868)				
	Sheet 2, filled by: Simran Jeet Gill (500122195)				
1	Aditya Khatri (500119453)				
2	Divya Israni (500121919)				
3	Amritanshi Singh (500119477)				
4	Anchit Saxena (500122868)				
	Sheet 3, filled by: Divya Israni (500121919)				
1	Aditya Khatri (500119453)				
2	Simran Jeet Gill (500122195)				
3	Amritanshi Singh (500119477)				
4	Anchit Saxena (500122868)				
	Sheet 4, filled by: Amritanshi Singh (500119477)				
1	Aditya Khatri (500119453)				
2	Simran Jeet Gill (500122195)				
3	Divya Israni (500121919)				
4	Anchit Saxena (500122868)				
	Sheet 5, filled by: Anchit Saxena (500122868)				
1	Aditya Khatri (500119453)				
2	Simran Jeet Gill (500122195)				

3	Divya Israni (500121919)				
4	Amritanshi Singh (500119477)				